

Lesson 3: More Python Skills

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Predict what code does
I Do	10–15 min	Classes, decorators, comprehensions, closures
We Do	15–20 min	Build a class + decorator together
You Do	5–10 min	List comprehension challenge
Wrap-Up	5 min	Assignment preview & Q&A

Warm-Up (5 min)

What Does This Code Do?

```
fruits = ["apple", "banana", "cherry"]  
result = list(filter(lambda f: "a" in f, fruits))  
print(result)
```

Take 30 seconds, then share your prediction in the chat.

I Do: Why Learn These Topics?

Today covers four "level-up" Python skills:

Concept	Why It Matters
Classes (OOP)	Organize data + behavior together
Decorators	Add behavior to functions without changing them
List comprehensions	Write loops in one clean line
Closures	Wrap data inside a function for safe keeping

These patterns show up everywhere in professional Python — especially in frameworks like Dash, Flask, and Django.

I Do: Classes — The Basics

A **class** bundles data and functions (called **methods**) together:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def speak(self):
        print("Bark!")
```

I Do: Classes — The Basics (Cont.)

```
dog1 = Dog("Spot", 2)
print(dog1.name)      # "Spot"
dog1.speak()          # "Bark!"
```

Key ideas:

- `__init__` runs automatically when you create an object
- `self` refers to the specific instance

I Do: What is `self`?

```
class Dog:
    def __init__(self, name, age):
        self.name = name    # store on THIS dog
        self.age = age

    def greet(self):
        print(f"Hi, I'm {self.name}!")
```

- `self` is always the **first parameter** in a method definition
- But you **don't pass it** when calling the method — Python does it automatically

```
dog1 = Dog("Spot", 2)
dog1.greet()    # Python passes dog1 as self
```

Think of `self` as "this object" — it gives methods access to the object's data.

I Do: Class Inheritance

Inheritance lets you build on an existing class:

```
class BigDog(Dog):  
    def speak(self):  
        print("W00F!")    # Override parent method  
  
    def fetch(self):  
        print("Got it!")  # New method  
  
dog3 = BigDog("Butch", 3)  
dog3.speak()    # "W00F!" (overridden)  
dog3.greet()    # "Hi, I'm Butch!" (inherited)  
dog3.fetch()    # "Got it!" (new)
```

BigDog gets everything **Dog** has, plus its own additions.

I Do: Decorators — The Concept

A **decorator** wraps a function to add extra behavior:

```
def my_decorator(func):  
    def wrapper():  
        print("Before!")  
        func()  
        print("After!")  
    return wrapper  
  
@my_decorator  
def say_hello():  
    print("Hello!")
```

I Do: Decorators — The Concept (Cont.)

```
say_hello()
```

Output:

```
Before!  
Hello!  
After!
```

`@my_decorator` is shorthand for: `say_hello = my_decorator(say_hello)`

I Do: Why Decorators?

Decorators add behavior without changing the original function.

A practical example — timing how long a function takes:

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"Finished in {elapsed:.4f} secs")
        return result
    return wrapper
```

`*args, **kwargs` makes the wrapper work with ANY function.

I Do: Decorators with Arguments

What if you want to pass arguments to the decorator itself?

```
def repeat(num_times):  
    def decorator(func):  
        def wrapper(*args, **kwargs):  
            for _ in range(num_times):  
                result = func(*args, **kwargs)  
            return result  
        return wrapper  
    return decorator  
  
@repeat(3)  
def say_hi():  
    print("Hi!")  
  
say_hi()  # Prints "Hi!" three times
```

Three levels: factory → decorator → wrapper

I Do: List Comprehensions

A Pythonic shortcut for building lists:

```
# Traditional way
squares = []
for x in range(5):
    squares.append(x ** 2)

# List comprehension
squares = [x ** 2 for x in range(5)]
# [0, 1, 4, 9, 16]
```

Add a filter:

```
even_squares = [x ** 2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]
```

Pattern: [expression for item in iterable if condition]

I Do: Closures

A **closure** is a function that remembers data from its outer scope:

```
def make_secret(secret):  
    def did_you_guess(guess):  
        if guess == secret:  
            print("You got it!")  
        else:  
            print("Nope!")  
    return did_you_guess  
  
game = make_secret("swordfish")  
game("magic")      # "Nope!"  
game("swordfish")  # "You got it!"
```

The inner function "closes over" the `secret` variable — it remembers it.

We Do: Let's Build a Point Class

The assignment asks you to build a Point class. Let's start it together.

What should a Point have?

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

What methods would be useful?

We Do: Adding Methods to Point

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Point({self.x}, {self.y})"

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def distance(self, other):
        return ((self.x - other.x)**2
                + (self.y - other.y)**2) ** 0.5
```

We Do: Adding Methods to Point (Cont.)

```
p1 = Point(3, 4)
p2 = Point(0, 0)
print(p1)           # Point(3, 4)
print(p1.distance(p2)) # 5.0
```

We Do: Extending Point → Vector

Now let's inherit from Point to make a Vector:

```
class Vector(Point):  
    def __str__(self):  
        return f"Vector({self.x}, {self.y})"  
  
    def __add__(self, other):  
        return Vector(self.x + other.x,  
                        self.y + other.y)
```

```
v1 = Vector(1, 2)  
v2 = Vector(3, 4)  
v3 = v1 + v2  
print(v3)           # Vector(4, 6)  
print(v3.distance(v1)) # Inherited from Point!
```

We got `distance()` for free through inheritance!

We Do: A Simple Decorator

Let's build a decorator that logs function calls — preview of Task 1:

```
def logger(func):  
    def wrapper(*args, **kwargs):  
        print(f"Calling: {func.__name__}")  
        print(f"  args: {args}")  
        print(f"  kwargs: {kwargs}")  
        result = func(*args, **kwargs)  
        print(f"  returned: {result}")  
        return result  
    return wrapper
```

We Do: A Simple Decorator (Cont.)

```
@logger
def add(a, b):
    return a + b

add(3, 5)
```

What will print?

You Do: List Comprehension Challenge (5 min)

Given this list of names:

```
names = ["Jazmine Rivera", "Luis Chen", "Amir Patel",  
         "Sara Edwards", "Tom Lee"]
```

Write list comprehensions for:

1. A list of just the first names

Hint: `name.split()[0]`

2. A list of names that contain the letter "e" (case-insensitive)

Hint: `"e" in name.lower()`

Bonus: Can you do both in one comprehension?

Assignment Preview

This week you'll create **6 separate files** — each focuses on a different concept:

Task	File	Concept
1	<code>log-decorator.py</code>	Logger decorator
2	<code>type-decorator.py</code>	Decorator with arguments
3	<code>list-comprehensions.py</code>	List comprehensions on CSV
4	<code>hangman-closure.py</code>	Closure-based hangman game
5	<code>extend-point-to-vector.py</code>	Class inheritance
6	<code>tictactoe.py</code>	Full class with exceptions

Assignment Tips

These tasks are independent — start with whichever feels most comfortable.

Task 1 (logger): Use `func.__name__` to get the function's name.

Task 4 (hangman closure): Very similar to the `make_secret` closure from the lesson — just add a `guesses` list.

Task 5 (Point → Vector): We just built most of this together!

Task 6 (tic-tac-toe): The assignment gives you `__str__`, `move()`, and `whats_next()`. You need to assemble the class and write the game loop.

Wrap-Up

Today we covered:

- Classes: `__init__`, `self`, methods, and inheritance
- Decorators: wrapping functions with extra behavior
- List comprehensions: concise list-building syntax
- Closures: functions that remember their outer scope

Quick check: On a scale of 1–5, how confident do you feel about writing a Python class?

Getting Help This Week

- Each task is a separate file — work on them **one at a time**
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** for help with decorators or classes
- **Read the provided code for Task 6** carefully — understanding it is half the work

Next week: Intro to Data Engineering with Pandas — the real fun begins!

You're Leveling Up! 🐍

Three weeks of Python fundamentals — strong foundation built.

See you next session!